前置知識與目標

- 前置知識為 BST、二分搜尋、有序序列、樹高與基本磁碟頁面概念。
- 能由 m-way search tree 推導 B-tree 佔用率及高度限制，追蹤分裂、借位與合併。
- 能區分 B-tree 與 B+ tree 的資料位置、separator 語義與範圍掃描成本。

多元搜尋樹的區間不變量

- 一個內部節點有遞增鍵 $k_1<\dots<k_r$，最多 m−1 鍵，並有 r+1 個子樹。
- 子樹區間依序是 $(-\infty,k_1)$、(k_1,k_2)、⋯、$(k_r,+\infty)$，並受祖先區間再限制。
- 單一節點可二分搜尋 r 個鍵，CPU 比較成本 $O(\log m)$。但一般 m-way tree 不保證平衡或最低佔用。
- 若節點可長期只有一個子或葉深度不同，不能僅因「每點最多 m 子」就宣稱高度 $O(\log_m n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

外部記憶體模型與 fanout

- 資料切成 block（區塊），一次 I/O 在主記憶體與外部儲存間搬一個 block。本章用 B 表示每塊可容納的項目量。
- 多個鍵與子指標放同頁，可在一次頁讀後決定下一層路徑。大 fanout（分支數）用來減少層數。
- 在分支及葉容量皆為Θ(B)的頁索引中，$O(1+\log_B n)$ 描述頁面 I/O，不代表每頁內只做 O(1) 次 CPU 比較。
- 實際頁大小、鍵長、指標、頁頭與填充率共同決定 fanout。SSD 也有頁與快取成本，但延遲不能由理論界直接推定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 採 minimum degree t 慣例

- 本教材 $t \geq 2$。非根節點含 $t-1$ 至 $2t-1$ 鍵。非葉內部節點有鍵數加一個子節點。
- 非葉根含至少 1 鍵、2 子。根若是葉，可為空。所有葉具有相同深度。
- 每個節點的鍵與子樹區間遵守搜尋次序。資料記錄可放在內部節點與葉，不限於葉。
- 若別的教材以「order m」表示最大子數，m 不一定與此 t 同義。比較公式前必須換算定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先看容量：order 3 與 minimum degree

這裡的 **order m** 指節點最多有 m 個子節點。非根節點的子數介於 $[m/2]$ 與 m，鍵數比子數少一；根與葉需依各自規則處理。

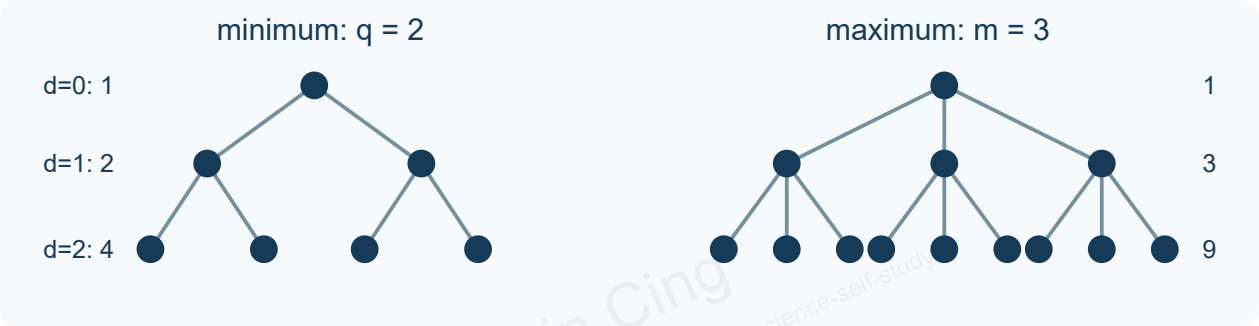
慣例	非根節點子數	非根節點鍵數	例子
order m	$[m/2]$ 至 m	$[m/2]-1$ 至 $m-1$	m=3：1–2 鍵
minimum degree t	t 至 2t	$t-1$ 至 $2t-1$	t=2：1–3 鍵

m=2t 時兩種容量相同；m 為奇數時沒有整數 t 讓上下限同時相同。2–3 tree 的 order 3 不能設 t=1.5，因本章 t 版本要求整數 $t \geq 2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一般 order m 的節點數上下界

令 q=⌈m/2⌉，非空樹的高度 h 以根到葉的邊數計。根至少 1 鍵；若 h≥1，根至少有 2 子，每個非根內節點至少 q 子。每層最少節點數依序為 1、2、2q、⋯、$2q^{h-1}$。



最少節點數為 $1+2(1+q+\cdots+q^{h-1})=1+2(q^h-1)/(q-1)$；所有非根節點至少 q−1 鍵，得到 $n \geq 2q^h-1$。h=0 時根為非空葉，也有 n≥1。

最大容量與 2–3 tree 數例

每層最多有 m 倍的節點；所有節點都放 m−1 鍵時，高度 h 的最大節點數是 $(m^{h+1}-1)/(m-1)$，最大鍵數為 $m^{h+1}-1$。

order	h（邊數）	最少節點	最多節點	最少鍵	最多鍵
3	2	7	13	7	26
4	2	7	21	7	63

例如 order 3、h=2 時，最少樹有根 1 個、下一層 2 個、葉 4 個；每個節點各 1 鍵，共 7 鍵。最大樹每層依序 1、3、9 個節點，每個節點 2 鍵，共 26 鍵。

若用「層數」H 計高度，H=h+1。以鍵數反推時，先用 $2q^h-1 \leq n \leq m^{h+1}-1$ 篩選可能的整數 h，再確認具體構造。

為何 order 3 要等溢位才分裂？

2–3 tree 的非根節點至少 1 鍵、至多 2 鍵。若把「已滿但未溢位」的 2 鍵節點先分裂，提升一鍵後只剩 1 鍵，無法給兩個非根子節點各 1 鍵。

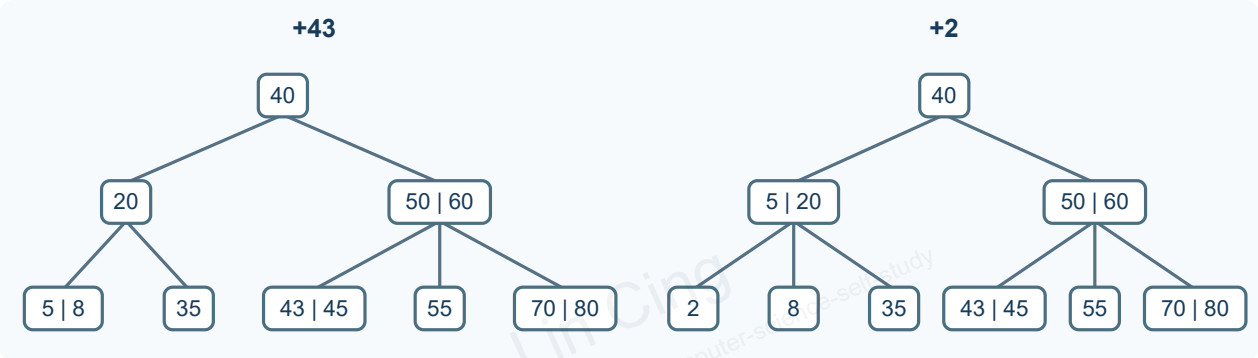
先插入新鍵，讓目標節點暫有 3 鍵；提升中間鍵，左右各留 1 鍵。父節點若因此暫有 3 鍵，再向上分裂。只有根分裂才讓樹增高。

這與 t=2 的 top-down 方法是兩套完整流程：後者的滿節點有 3 鍵，可先提升 1 鍵並讓兩側各留 1 鍵，再下降插入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

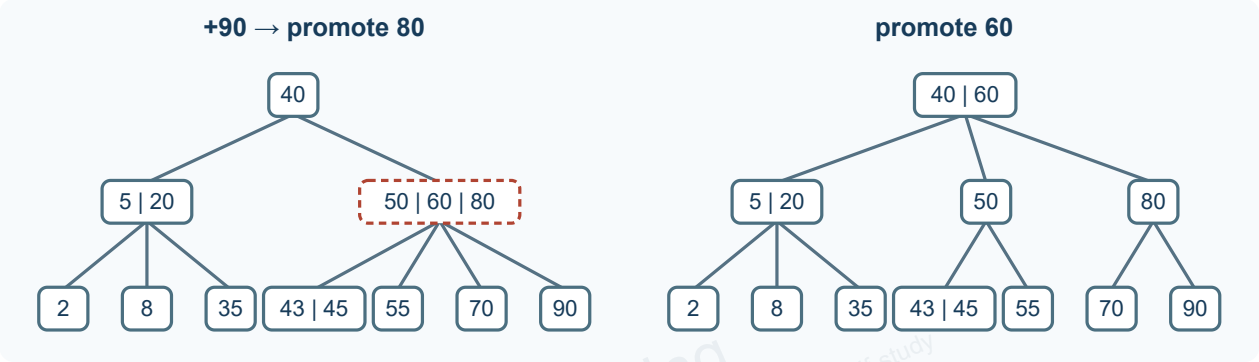
2–3 tree 插入：先處理 43 與 2

初始根 [40]；左內節點 [20] 有葉 [5,8]、[35]，右內節點 [50,60] 有葉 [45]、[55]、[70,80]。依序插入 43、2、90。



43 使葉 [45] 變成 [43,45]。插入 2 後 [5,8] 暫成 [2,5,8]；將 5 提至父節點 [20]，兩個新葉分別為 [2] 與 [8]。鍵 5 移到父節點，不再留在 B-tree 葉中。

插入 90：分裂向上傳遞



葉 [70,80] 插入 90 後成 [70,80,90]；提升 80，父 [50,60] 暫成 [50,60,80]。再把 60 提到根：左內節點 [50] 帶四個子區間中的前兩個，右內節點 [80] 帶後兩個；根成 [40,60]。

最後每個非根節點有 1 或 2 鍵，所有葉同深。只增加 43、2、90；提升是移動資料鍵，不是複製一筆資料到父頁。

B-tree 高度上界推導

- 對高度 $h \geq 1$ 的非空樹，根至少有 2 子，此後每個非根內點至少 t 子。
- 深度 $d \geq 1$ 至少 $2t^{d-1}$ 個節點，各至少 $t-1$ 鍵。
- 因此 $n \geq 1+2(t-1)\Sigma_{d=1}^{h} t^{d-1}=2t^h-1$，得到 $h \leq \log_t((n+1)/2)$。
- t 與 block 容量同階且一點占一塊時，搜尋只走一條根葉路徑，I/O 為 $O(1+\log_B n)$，根快取可再少一次讀取。

搜尋同時檢查節點鍵與子區間

1. 在節點內找第一個不小於 x 的鍵位置 i。
2. 若恰等於 x，回傳該記錄。若未找到且是葉，回報不存在。
3. 否則進入第 i 個子樹，重複上述步驟。

根鍵 `[20,50]` 將查詢 37 導向中間子樹。B-tree 在內部鍵相等時可以成功，這點與本教材 B+ tree 的葉端查詢不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 插入先分裂滿子節點

- 採 top-down 插入。若根滿，先建立新根並分裂舊根，再下降。
- 滿節點有 $2t-1$ 鍵，中位第 t 鍵提升到父。左右各保留 $t-1$ 鍵及相應子指標。
- 每次下降前先分裂滿子，保證抵達葉時有空位。父節點此時也必須有空位，這由下降不變量保證。
- 分裂保持鍵總集合與葉深度一致。根分裂是唯一使整體高度增加一層的事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

t=2 插入 trace

- 依序插入 `10,20,5`，根葉為 `[5,10,20]`，已滿。
- 插入 6 前分裂根，提升 10，左右葉為 `[5]`、`[20]`。再把 6 插入左葉得 `[5,6]`。
- 繼續插入 `12,30`，右葉成 `[12,20,30]`。插入 25 前分裂該葉，把 20 提升到根。
- 最終根 `[10,20]`，三葉 `[5,6]`、`[12]`、`[25,30]`。鍵 10 與 20 在內部，不重複留在葉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 分裂與插入的完整陣列操作

```
split_child(parent,i):
    y=parent.children[i]; require len(y.keys)==2*t-1
    z=Node(leaf=y.leaf,keys=[],children=[])
    median=y.keys[t-1]
    z.keys=y.keys[t:]; y.keys=y.keys[:t-1]
    if not y.leaf:
        z.children=y.children[t:]; y.children=y.children[:t]
    insert median at parent.keys[i]
    insert z at parent.children[i+1]
insert_nonfull(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k: return already_present
    if x.leaf: insert k at i; return
    if x.children[i] is full:
        split_child(x,i)
        if k>x.keys[i]: i+=1
        else if k==x.keys[i]: return already_present
    insert_nonfull(x.children[i],k)
```

根滿時先以新根含舊根一子，split_child(newroot,0)，才呼叫insert_nonfull。內部節點需同時分配children，不能只分鍵；左t子和右t子各對應t−1鍵，葉深度因新根提升才整體加1。

B-tree 分裂與插入的完整陣列操作 （續）

t=2的本章序列最後根[10,20]、葉[5,6],[12],[25,30]。再插7只改左葉為[5,6,7]；再插8前，分裂該葉提升6，根成[6,10,20]，葉[5],[7,8],[12],[25,30]。父此時滿但本次已完成；下一次插入前才分裂根。這展示top-down滿子分裂與「溢位後才分裂」是不同成套程序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除前確保下降子節點有至少 t 鍵

- 若欲下降的子只有 t−1 鍵，先透過借位或合併補強，避免刪除後低於最低佔用。
- 若相鄰兄弟有至少 t 鍵，從兄弟經父 separator 旋轉一鍵，必要時一併移動子指標。
- 若兩相鄰子都只有 t−1 鍵，連同父的一個 separator 合併，得到 $2t-1$ 鍵。
- 借位不改變層高。合併可能使父變少；top-down 的下降條件使非根父仍合法，根有特例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除內部鍵與縮根

- 若鍵在內部節點，左子至少 t 鍵時，可用其前驅取代，再從左子刪前驅。
- 否則若右子至少 t 鍵，使用後繼；若兩子都只有 t−1 鍵，與目標鍵合併，再往合併子刪除。
- 若根刪到 0 鍵而仍有一子，讓該子成為新根，高度減 1。若空葉根無資料，整樹為空。
- 每層至多常數次節點重組，I/O `O(1+log_t n)`；若節點內以陣列搬動 t 個鍵，CPU 時間可為 `O(t(1+log_t n))`。

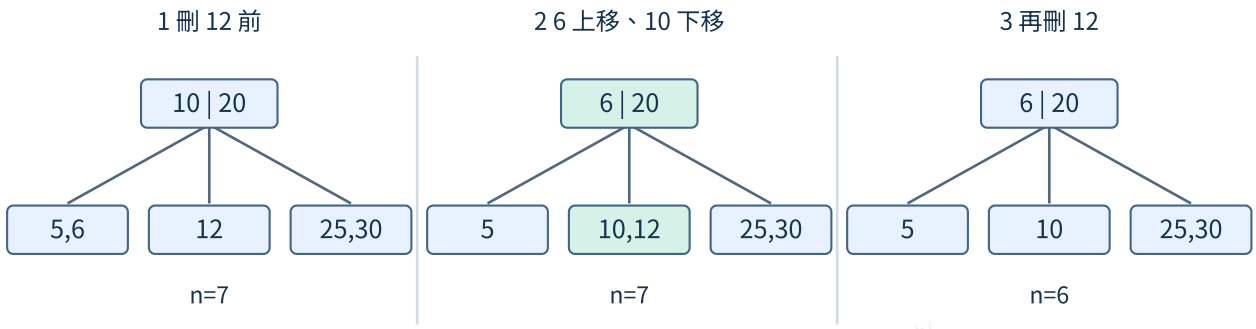
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除借位 trace

- 從根 `[10,20]`、葉 `[5,6]`、`[12]`、`[25,30]` 刪除 12，取 t=2。
- 中間葉只有 1 鍵，下降前向左兄弟借位。父的 10 下移到中葉，左兄弟最大鍵 6 上升成父鍵。
- 狀態變成根 `[6,20]`，葉 `[5]`、`[10,12]`、`[25,30]`。再刪 12 得中葉 `[10]`。
- 所有非根節點至少 1 鍵，葉深度一致，中序鍵集合正好移除 12。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 借位時資料鍵的雙向移動



t=2，每個非根葉需 1 至 3 鍵。下降到 [12] 前，先把父資料鍵 10 放進中葉，同時把左兄弟最大資料鍵 6 放進父的原槽位；這是兩筆不同資料的移動，不是把 6 複製到父後仍留在左葉。第二態中葉已含兩鍵 [10,12]，才可刪除 12，得到第三態。

三態的中序依次為 [5,6,10,12,20,25,30]、同一序列、[5,6,10,20,25,30]。借位不遺失或重複記錄，刪除才恰少一筆；葉深度都為 1。若兄弟是內部節點，連同邊界子指標移動，不能只照葉圖搬鍵。B+ 的父 separator 只是路由副本，不能把此 B-tree 資料移動規則套給 B+ 葉借位。

B-tree 刪除借位、合併與縮根的控制流程

```
prepare_child(x,i):
    child=x.children[i]
    if len(child.keys)>=t: return i
    if i>0 and len(x.children[i-1].keys)>=t:
        left=x.children[i-1]
        child.keys.insert(0,x.keys[i-1]); x.keys[i-1]=left.keys.pop()
        if not child.leaf: child.children.insert(0,left.children.pop())
    else if i+1<len(x.children) and len(x.children[i+1].keys)>=t:
        right=x.children[i+1]
        child.keys.append(x.keys[i]); x.keys[i]=right.keys.pop(0)
        if not child.leaf: child.children.append(right.children.pop(0))
    else:
        if i>0: merge_children(x,i-1); return i-1
        merge_children(x,i); return i
    return i
```

若目標在葉直接移除；在內部時，左子至少t鍵便以前驅替換並遞迴刪前驅，否則右子至少t鍵便用後繼，兩子都t−1則合併目標與兩子再向下。若目標不在當前節點，先prepare_child再用回傳的索引下降；向左合併後原索引會減1，不能沿用舊i。結束後根空且有一子便縮根。

B-tree 刪除借位、合併與縮根的控制流程 （續）

本章刪12的左借例為根[10,20]、中葉[12]，左[5,6]借6上父、10下中，刪後根[6,20]、葉[5],[10],[25,30]。再刪5：左葉只有1鍵，右鄰中葉也1鍵，合併[5]+父6+[10]得[5,6,10]，根剩[20]，再刪5得[6,10]。若接著把右葉[25,30]刪25成[30]、左刪6成[10]，再刪10前需合併[10]+20+[30]，根空而縮為新葉[10,20,30]，刪後[20,30]。

每次借位保持排序與同葉深；合併容量2t−1恰合法。節點一頁時每層常數I/O，總O(1+log_t n)；頁內陣列位移可O(t)每層，不能把頁I/O界當CPU步數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 根入口與完整刪除遞迴

```
insert(k):
    if root is full:
        old=root; root=Node(leaf=false,keys=[],children=[old])
        split_child(root,0)
    insert_nonfull(root,k)
erase_from(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if x.leaf: remove x.keys[i]; return
        if len(x.children[i].keys)>=t:
            p=maximum_key(x.children[i]); x.keys[i]=p
            erase_from(x.children[i],p)
        else if len(x.children[i+1].keys)>=t:
            s=minimum_key(x.children[i+1]); x.keys[i]=s
            erase_from(x.children[i+1],s)
```

B-tree 根入口與完整刪除遞迴 （續）

```
        else:
            merge_children(x,i); erase_from(x.children[i],k)
    else:
        if x.leaf: return absent
        i=prepare_child(x,i); erase_from(x.children[i],k)
erase(k):
    erase_from(root,k)
    if len(root.keys)==0 and not root.leaf:
        old=root; root=old.children[0]; release(old)
merge_children(x,i):
    left=x.children[i]; right=x.children[i+1]
    left.keys+= [x.keys[i]] + right.keys
    if not left.leaf: left.children+=right.children
    remove x.keys[i] and x.children[i+1]; release(right)
```

空B-tree以一個0鍵葉根表示，因此空樹erase會在葉回報absent。此頁簡寫key；key-value記錄實作需搬動整筆配對，不能只改鍵。merge_children只用於兩子各t−1鍵的狀態，合併後(t−1)+1+(t−1)=2t−1鍵，恰未超容量。父的中間鍵真的是資料記錄，與B+內部純separator不同。

B-tree 根入口與完整刪除遞迴 （續）

下降不變量是非根遞迴入口至少t鍵；prepare_child先借位或合併維持此條件。命中內部鍵的前驅／後繼分支只選至少t鍵的子，刪後仍至少t−1；兩子都不能借時，把目標一起合併才繼續刪。prepare_child若往左合併必回傳i−1，因舊child[i]已被釋放。即使鍵不存在，也可能先重組路徑；抽象鍵集合不變，不承諾不存在時實體樹形不變。

根[20]、葉[10],[30]，t=2，刪10前兩子都只有一鍵，merge_children把根20夾在中間，得葉[10,20,30]；刪10後[20,30]，空內部根縮成此葉。若錯把separator20直接丟掉，會遺失尚未刪的資料。葉深一致性由同層兄弟重組保留，只有根合併改變全樹共同高度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B+ tree 把資料記錄留在葉

- 本章 B+ tree 的資料 key-value 全在葉，內部節點只放 separator 與子指標。
- 內部 separator 定義為右子樹最小鍵，查詢相等時往右側下降。此慣例與「取左最大」不可混用。
- 葉依鍵序連結，讓範圍查詢在定位首葉後沿葉鏈掃描。
- 內部 separator 是路由資訊，可能與葉內鍵重複，不能把內部與葉鍵數相加當成資料筆數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B+ 容量必須分別定義

- 本教材取整數 $M \geq 3$、$L \geq 2$。非根內部節點子數範圍 $[a, M]$，其中 $a = \lceil M/2 \rceil \geq 2$；內部根有 2 至 M 子，內部鍵數為子數減一。最低分支至少 2 才能支援後續對數高度論據。
- 非根葉容量範圍 $[\lceil L/2 \rceil, L]$ 筆記錄。根為葉時允許 0 到 L 筆。
- 內部 M 與葉 L 可不同，因內部儲存子指標，葉可能存 record pointer 或完整資料。
- 葉分裂後把右葉最小鍵複製到父。內部分裂則提升分隔邊界並重新分配子節點，不能直接照搬葉分裂公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B+ 葉分裂與 separator

- 設 L=3，原葉 `[4,9,12]`，插入 10 形成暫時序列 `[4,9,10,12]`。
- 分成左葉 `[4,9]`、右葉 `[10,12]`，把 separator 10 複製到父，並接好前後葉鏈。
- 若父只有兩個子，根為 `[10]`。查 10 時必須向右，直到右葉讀到真正記錄。
- 與 B-tree 不同，10 不因提升到父而從葉移除。

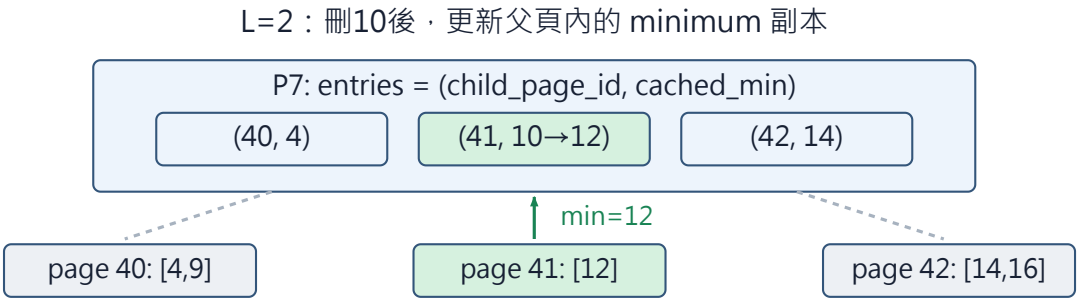


磁碟頁布局與局部 minimum 更新

```
// Internal page: entries[j]=(child_page_id,cached_min), all in this page.
// Separators are entries[1:].cached_min; no persistent parent pointers.
refresh_local(p):
    if p.leaf: p.minimum=p.records[0].key if nonempty else absent
    else: p.minimum=p.entries[0].cached_min
entry(p): return (p.id,p.minimum)
find_leaf(k):
    path=[]; p=read(root_id)
    while not p.leaf:
        i=upper_bound(p.entries[1:].cached_min,k)  // equality goes right
        path.push((p.id,i)); p=read(p.entries[i].child_page_id)
    return path,p
```

磁碟模型中，每個節點恰占一頁。內頁將每個子頁的ID與其精確minimum副本放在本頁；separator只是第1筆以後的cached_min，不需再讀子頁。葉頁存有序記錄與next/prev頁ID。minimum副本由被修改子頁回傳後，沿祖先stack增量維護；未變動entry保持原值。

磁碟頁布局與局部 minimum 更新 （續）



灰虛線為 page-id 連結；綠箭頭回傳已讀葉的 minimum。
只讀 P7 與葉41；separator [12,14] 由父頁本地 min 副本重建。

圖採M=3、L=2，父P7的三筆entry原為(40,4),(41,10),(42,14)，葉41為[10,12]。刪10後葉41剩[12]仍達最低1筆，只需將P7的entry1改為(41,12)，separator由[10,14]變[12,14]。頁40、42的資料沒有改，cached_min也不改；不必讀它們。

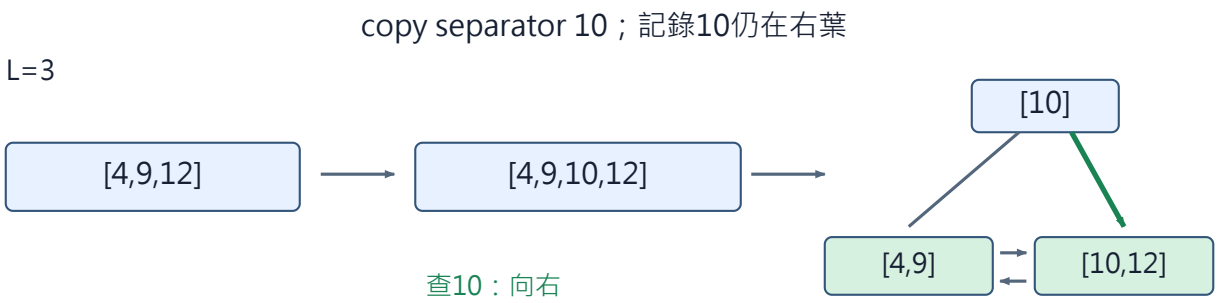
若refresh逐一讀M個child.minimum，即使只改一筆也可能多M次冷頁讀取；若internal split搬Θ(M)個child後逐頁改parent，也會多Θ(M)次I/O。把欄位稱作「快取」並不讓它自動在父頁。本布局明定這些副本的實際位置，並完全取消持久parent；祖先stack只存(page_id,index)，向上重讀父頁即可。

這與Open Data Structures § 14.2.2避免B-tree子頁parent改寫的設計理由一致。

B+ 插入、精確separator與向上分裂

```
insert(k,v):
    path,node=find_leaf(k)             // stack holds (parent_page_id,index)
    insert/update (k,v) in sorted node.records
    while true:
        if node exceeds L records or M entries:
            parts=split_into_floor_ceil_halves(node)
            if node.leaf: splice both leaf links and repair adjacent leaf
        else: parts=[node]
        for p in parts: refresh_local(p); write(p)
        if path empty:
            if len(parts)==1: root_id=parts[0].id
            else:
                r=new_internal(entries=[entry(p) for p in parts])
                refresh_local(r); write(r); root_id=r.id
            return
        (parent_id,i)=path.pop(); parent=read(parent_id)
        parent.entries[i:i+1]=[entry(p) for p in parts]
        node=parent
// entry(p)=(p.id,p.minimum); internal splits move complete cached entries.
```

B+ 插入、精確separator與向上分裂 （續）



L=3的[4,9,12]插10→[4,9,10,12]→葉[4,9],[10,12]，根[10]。再插14使右葉[10,12,14]；插16使右葉分成[10,12],[14,16]，父根變[10,14]。10與14在父是路由副本，各資料記錄仍在葉，根分隔區間是(-∞,10)、[10,14)、[14,+∞)。

此處採前節定義的磁碟頁布局：內頁 entries 同頁保存 (child_page_id,cached_min)，不在子頁存 parent。左半沿用原page-id，右半配置新頁；分裂只搬完整 entry，既有 cached_min 隨它一起移動；左右新頁各由本頁資料 refresh_local，父頁只接收已變動子頁回報的新 entry。不讀其他子頁來重算 minimum，也不逐一改 moved child 的磁碟 parent。

B+ 插入、精確separator與向上分裂 （續）

find_leaf 保存的是祖先頁ID與下降索引，不保留全部祖先頁。向上時重讀當前父頁，以一或兩筆新 entry 取代原 entry；父頁若溢位再分裂，根溢位才建新根。沒有溢位也繼續傳播精確 minimum，故首鍵變更不會在上層留下舊 separator。非根內頁至少a=ceil(M/2)子，葉至少 ell=ceil(L/2)筆，M≥3,L≥2確保兩半合法。

每層只讀寫常數個頁；葉的雙向鏈至多另更新相鄰葉一頁。因此本布局插入為O(h+1)頁I/O，頁內陣列搬移則最壞O(L+Mh) RAM工作。祖先stack為O(h+1)個詞，另外需常數個大小O(M+L)的頁緩衝。完整記錄大小、entry大小與頁容量須先固定，不能把RAM與磁碟欄位任意混用。

B+ 刪除與邊界更新

- 葉刪除後低於最低記錄數，先嘗試從相鄰葉借記錄，否則合併並移除父的 separator。
- 即使沒有 underflow，只要右子樹最小鍵改變，採精確「右最小」separator 的本教材就需更新相應祖先邊界。
- 原例從右葉 `[10,12]` 刪除 10，若 L=2、最少 1 筆，葉不缺額，但父 separator 應由 10 改成 12。
- 某些系統允許保留舊但仍合法的 fence key，規則不同。不能一邊採精確右最小定義，一邊說父鍵永不需要更新。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B+ 刪除的借位、合併與範圍讀取

```
erase(k):
    path,node=find_leaf(k)
    if k absent: return
    remove record k from node
    while true:
        refresh_local(node)
        if path empty:
            if node internal and len(node.entries)==1:
                root_id=node.entries[0].child_page_id; free(node)
            else: write(node)
            return
        (parent_id,i)=path.pop(); parent=read(parent_id)
        if node meets its nonroot minimum occupancy:
            write(node); parent.entries[i]=entry(node)
        else:
```

B+ 刪除的借位、合併與範圍讀取 （續）

```
            read adjacent sibling(s) of the same parent
            if a sibling can lend:
                move a boundary record or complete cached entry
                refresh_local both; write both; update their parent entries
            else:
                merge adjacent records or cached-entry arrays
                repair both leaf links if leaves
                refresh_local/write merged page; free the other page
                replace their two parent entries by entry(merged)
        node=parent
range(lo,hi):
    locate first record>=lo
    scan leaf links until key>hi or leaf chain ends
```

取L=3，根[10,14]，葉[4,9],[10,12],[14,16]，最低2筆。刪10使中葉[12]缺額；兩兄弟各2不能借，與左葉合併為[4,9,12]，根只剩[14]，葉鏈接[4,9,12]↔[14,16]。再刪14使右葉[16]缺額；左葉有3可借，移12到右葉前，得到[4,9]↔[12,16]，根separator更新12。若只刪右葉最小但未缺額，精確separator仍要更新。

B+ 刪除的借位、合併與範圍讀取 （續）

借位／合併都移動完整本頁entry；不讀被移動的子頁，也不改子頁parent。借位後仍沿保存stack傳播minimum，不能因結構缺額消失而漏掉祖先separator更新。內頁合併最多2a−1≤M個entry，葉合併最多2ell−1≤L筆；同層合併保留葉深。根若只剩一子，直接採其page-id為root。每層常數個父／兄弟／相鄰葉頁I/O，總O(h+1)，RAM搬移最壞O(L+Mh)。

區間[9,14]在最後狀態先定位左葉9，再沿next讀12，遇16>14停止，輸出[9,12]。以一般高度h與非根葉最低ell筆計，索引I/O為O(h+1+k/ell)；只有M、L都與block容量B同階時，才化成O(1+log_B n+k/B)。若葉只存record pointer且資料分散，再加D個資料頁，最壞D=k。Θ(k)輸出工作另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

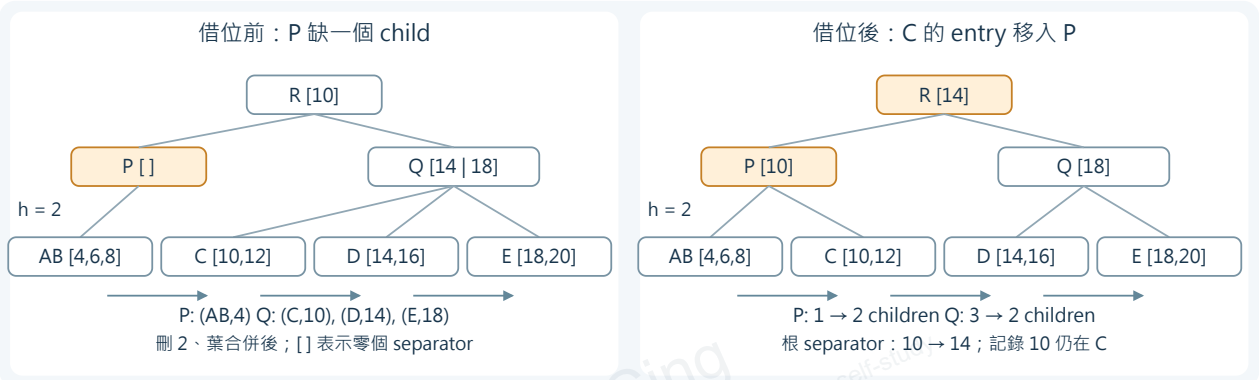
內部缺額：先看 child 數，不只看 separator

取 M=4、L=3：非根內頁至少 2 個 child，非根葉至少 2 筆記錄。根 R 有子頁 P、Q；P 的葉為 A=[2,4]、B=[6,8]，Q 的葉為 C=[10,12]、D=[14,16]、E=[18,20]。所有葉深度為 2；R 的 separator 是 10。

步驟	葉與內頁的實際變化	必須繼續處理的原因
刪除 2	A 只剩 [4]	1 筆低於葉最低 2 筆
檢查 B	B=[6,8]，不能出借	出借後 B 也只剩 1 筆
合併 A、B	保留 A 的頁 ID，改稱 AB=[4,6,8]；釋放 B	3 筆恰好不超過 L
更新 P	P.entries=[(AB,4)]；P.minimum=4	**只有 1 個 child，低於內頁最低 2 個**

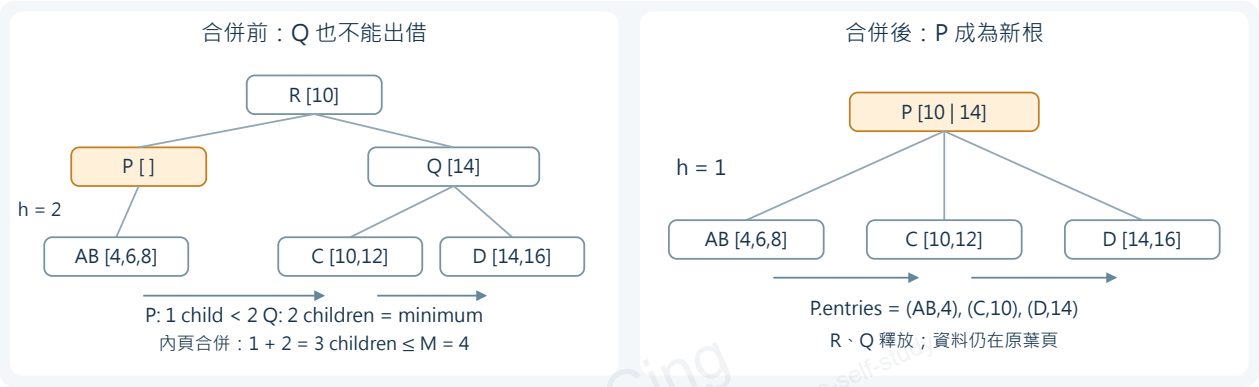
P 沒有 separator，卻仍是一個指向葉 AB 的內頁，不是空葉。葉鏈由 A↔B↔C 改成 AB↔C；C.prev 改指向 AB。根暫時仍含 (P,4)、(Q,10)。這是一次操作中的暫態，修好 P 後才交付合法索引。

內頁借位：移動整筆 entry，葉資料留在原頁



Q 原有 3 個 child，可以把最左的 (C,10) 移到 P 尾端。P 變成 [(AB,4),(C,10)]，Q 變成 [(D,14),(E,18)]；兩者各有 2 個 child，達到最低佔用。P 的 separator 為 10，Q 的 separator 為 18，根由 [(P,4),(Q,10)] 改成 [(P,4),(Q,14)]。

兄弟不能借：合併內頁，再收縮根



另取一棵沒有 E 的初始樹，其他容量與 A、B、C、D 不變。刪 2、合併 A/B 後，P 只有 (AB,4)，Q 只有 (C,10)、(D,14)，不能出借。串接兩頁的 entry 得 [(AB,4),(C,10),(D,14)]，保留 P、釋放 Q；P 的 separators 由本頁副本重建為 [10,14]。

R 只剩 (P,4) 一個 child，故令 root_id=P、釋放 R，高度由 2 降為 1。

用搜尋與葉鏈核對重組結果

兩個分支各自只刪除記錄 2。內頁移動的是頁 ID 與 minimum 副本；C 的實體資料仍是 [10,12]。父 separator 10 不是另一筆待插入葉的資料。

檢查	內頁借位後	內頁合併、縮根後
內頁與高度	R=[14]；P=[10]、Q=[18]；h=2	新根 P=[10,14]；h=1
查 10	R 向左到 P，相等向右到 C	P 在 10 處相等向右到 C
葉鏈	AB↔C↔D↔E，共 9 筆記錄	AB↔C↔D，共 7 筆記錄
葉記錄 10	只在 C 出現一次	只在 C 出現一次

在縮根分支查閉區間 [8,15]：先在 AB=[4,6,8] 找到 8，沿 next 到 C=[10,12]，再到 D=[14,16]。依序輸出 8、10、12、14；檢查到 16>15 時停止。圖中箭頭只畫 next，prev 指向同一條鏈的前一頁。內頁重組沒有改動這條葉鏈。

為什麼內頁修復不遺失資料，也不增加樹高？

內頁 entry 按子樹最小鍵排序。向右兄弟借最左 entry，等於把有序子樹序列的分界右移一格；向左兄弟借最右 entry 則相反。子樹內部不變，所有 child 深度相同，因此排序與葉深度都保留。精確 separator 直接由新 entry 序列重建，祖先 minimum 副本則繼續向上更新。

令 a=⌈M/2⌉。缺額內頁有 a−1 個 child，不能出借的兄弟有 a 個；合併後為 2a−1≤M 個 child。M 偶數時為 M−1，奇數時為 M。本例 1+2=3≤4。**B+ 合併串接 child entries，不額外插入父 separator 作為資料**；新內頁的 separator 數自然是 child 數減一。

每次向上只處理當前頁、父頁與至多兩個相鄰兄弟；葉合併另修常數個葉鏈欄位。本章沒有持久 child.parent，搬 entry 不必改寫被搬動的子頁。故每層常數頁 I/O，最壞 O(h+1)；這是漸近界，精確讀寫數仍取決於哪些頁已在 buffer（頁面緩衝）中。只有收縮根會使所有葉共同減少一層。

B+ 高度的分支與葉容量前提

```
a=ceil(M/2) >= 2; ell=ceil(L/2) >= 1
h=number of edges on a root-to-leaf path
if h>=1:
    leaf_count >= 2*a^(h-1)
    n >= 2*ell*a^(h-1)
    h <= 1+log_a(n/(2*ell))
// Root-only trees h=0 cost one page, including the empty tree.
```

根為內頁時至少2子，之後每個非根內頁至少a子。因此深度h至少2a^(h−1)個葉；每葉至少ell筆，得到上式。根是葉的h=0情形單獨處理。一般定位成本為h+1頁，故可統一寫O(1+log_a(1+n/ell))，不能只由M≥3、L≥2便把底數換成B。

反例取任意整數q≥3，M=3、L=B=2^q，所有內頁恰2子，所有2^q個葉各滿載B筆。樹高h=q、記錄n=B^2，合法冷頁點查需q+1頁；log_B n卻恆等於2。當q增長，O(log_B n)常數界不成立。失敗原因是內頁最低分支固定2，沒有隨B增長。

只有另假設固定大小的key／entry／record、一節點一頁，且M=Θ(B)、L=Θ(B)，才有a、ell=Θ(B)，高度化為O(1+log_B n)。一般range回傳k筆時，除首尾可能只取部分頁，其餘掃過的葉至少含ell筆輸出，因此索引I/O為O(h+1+k/ell)。加上上述兩個容量條件，才成O(1+log_B n+k/B)。

點查與範圍查詢 I/O

根葉定位為O(h+1)頁，一般h界見上一節，不預設M、L隨B增長。範圍查詢的索引頁成本為O(h+1+k/ell)。只有M=Θ(B)、L=Θ(B)、每點一頁及固定項目大小，才可寫O(1+log_B n+k/B)。

葉若僅存record pointer，完整資料另讀D個資料頁，最壞D=k；聚集且每頁Θ(B)筆時，資料頁可再受O(1+k/B)控制。輸出k筆仍需Ω(k)基本輸出工作，頁I/O與CPU工作是不同分母。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

理論 B+ 與資料庫實作

- PostgreSQL 18 官方文件描述多層 B-tree 索引，各層頁面有雙向鏈，葉項目指向資料列，內部項目指向下一層頁。
- 這支持「多路頁面索引」的實務動機，但不是本教材簡化刪除演算法的逐步實作保證。
- 真正資料庫另需並發控制、分裂中的讀取一致性、WAL 與崩潰恢復。單執行緒指標圖沒有處理這些問題。

參考資料與署名

Lin Cing，《資料結構》手寫筆記。 Pat Morin, Open Data Structures, § 14.2. PostgreSQL 18, B-tree implementation.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study